

实践4：蹲姿行走策略，G1速度+骨盆高度MDP设计

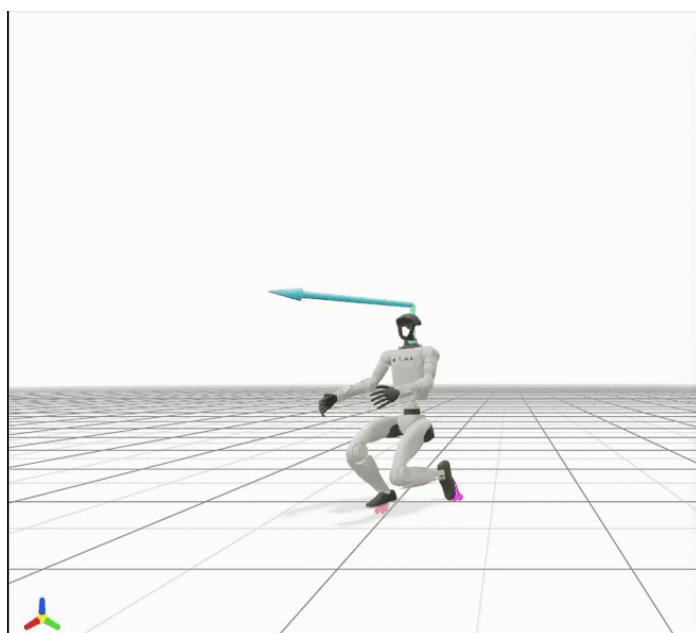
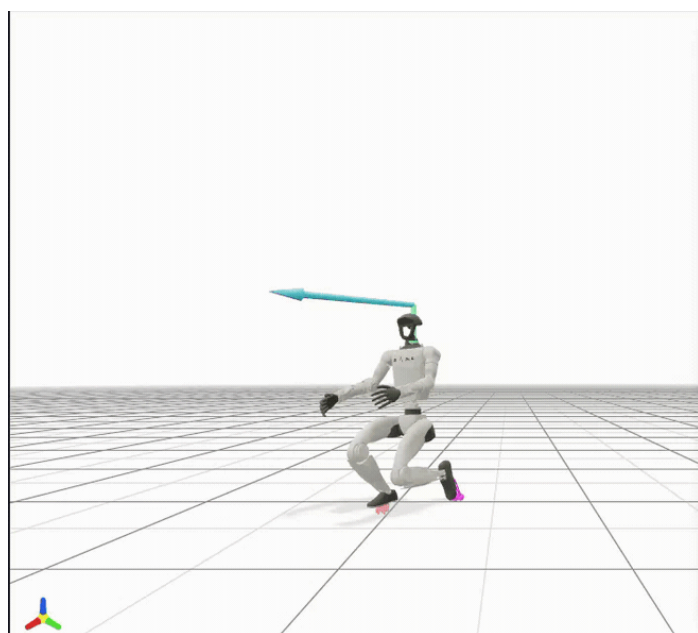


深蓝学院《人形机器人运动控制》课程第3章

课程链接：<https://www.shenlanxueyuan.com/course/864>

1. 作业目标

本作业对应课程第3章理论。本作业在 **mjlab** 框架中完成 **MDP 设计** → **并行训练** → **Viser 可视化验证** 的完整 RL 工作流，为 **Unitree G1** 设计并实现 **速度 + 骨盆高度** 双指令 MDP，训练策略使其能在平地行走的同时蹲低或站高。



为什么需要掌握 mjlab? mjlab 是面向人形/足式机器人的 **轻量级** RL 训练框架，底层基于 **MuJoCo** 物理引擎与 **NVIDIA Warp** GPU 并行。MuJoCo 在接触动力学建模上精度高，Warp 则支持在单块 GPU 上同时 rollout 并行环境，大幅缩短策略训练时间。近年来 RSS、CoRL、RA-L 等 **人形机器人** 方向的顶会/期刊工作中，越来越多团队采用 mjlab 或其上游生态进行 locomotion 研究——掌握 mjlab 的 **任务注册**、**MDP 配置**、**train/play 命令** 是进入该方向研究的实用技能。

主任务（完成全部 TODO 后训练）：

代码块

```
1 cd mjlab
2 uv run train Mjlab-VelocityHeight-Flat-Unitree-G1 --env.scene.num-envs 4096
```

完成本作业后，你应当能够：

能力项	说明
掌握 mjlab 工作流	会使用 <code>list-envs</code> / <code>train</code> / <code>play</code> ，理解任务 ID、env cfg、MDP manager 的配置方式
理解双指令 MDP	掌握 velocity + base_height 联合指令空间的设计与采样
实现高度指令逻辑	在 <code>height_command.py</code> 中完成均匀采样与跟踪 metrics
完成环境接线	在 <code>env_cfgs.py</code> 中注册命令、观测与奖励项
实现高斯跟踪奖励	编写 <code>track_base_height</code> 及速度跟踪奖励的误差与 return
奖励四维分类	理解 Task / Style / Reg / Penalty 分类，并在代码与报告中逐项阐述设计意图
补全课程与特权观测	速度课程学习三阶段 + Critic 足部接触观测
实验报告与消融	撰写设计思路、权重依据、≥2 组消融对比及训练曲线
Viser 交互验证	在 h = 0.5 m 下验证蹲姿行走 ≥ 10 s， 强烈建议 录屏作为性能佐证

作业边界（重要）：

- 10 处 TODO 为必做实现（能否训练与提交的前提）；最终成绩按评分细则的四个维度评定，**不按 TODO 逐项给分**
- 配置中 weight / std **已预填**；代码侧以完成 TODO 逻辑为主，**不强制**在代码里精细调参
- 须在 **实验报告** 中说明各项奖励的设计意图与权重选择依据，并完成 **≥ 2 组**消融对比（见 § 8.2）
- `variable_posture`、速度 `command` 观测等 **已实现内容** 供阅读，**不是** 本作业 TODO

2. 作业背景（理论）

本节介绍 **蹲姿行走** 任务的 MDP 设计，以及本作业所使用的 **mjlab / Viser** 工具链。请先通读本节，再阅读环境配置与 TODO 实现——理论中的每个公式，都将在后续内容汇总有直接的代码对应。

符号说明：下文公式均用纯文本表示，变量与代码/config 的对应关系见各节表格。

2.0 仿真与训练工具链：mjlal 与 Viser

本作业不仅要求你 **实现 MDP 公式**，还要求你 **熟悉 mjlal 的使用方式**——后续课程与科研中，你将反复用到「改 cfg → 训练 → play 验证」这一流程。

mjlal 是什么？

mjlal 是基于 **MuJoCo** 的轻量级 RL 环境框架，采用 **ManagerBased** 设计：命令（Command）、观测（Observation）、奖励（Reward）、课程（Curriculum）等 MDP 组件各自注册为独立 term，由 manager 统一调度。本作业的 10 个 TODO 正是围绕这一架构展开。

特性	说明	与本作业的关系
MuJoCo 动力学精度	接触、摩擦、关约束建模成熟	蹲姿行走依赖足地接触，仿真可信度直接影响策略质量
NVIDIA Warp 并行	GPU 上批量运行数千个独立 MuJoCo 实例	默认 4096 envs 训练，数小时即可收敛
轻量、可定制	纯 Python 配置，无重型 IDE 依赖	你修改的 <code>env_cfgs.py</code> 、 <code>rewards.py</code> 等即 MDP 定义
人形研究生态	多篇 RSS / CoRL / RA-L 人形 locomotion 工作采用类似栈	本作业是进入该生态的 最小可行练习

与实践 3 手写 MuJoCo 循环相比，mjlal 将 **仿真步进、并行采样、PPO 更新、日志与 checkpoint** 封装为 `uv run train` 一条命令；你的核心工作是 **定义 MDP**（指令、观测、奖励），而非重写训练循环。

Viser 是什么？本作业为何使用它？

Viser 是 Web 端 **3D 交互可视化** 库。mjlal 的 `play` 命令支持 `--viewer viser`，在浏览器中打开仿真视图，并提供 **GUI 控件**：

- **速度 slider** — 实时改变 `(v_x, v_y, ω_z)` 指令，观察行走/转向
- **高度 slider** — 实时改变 `h_cmd`，观察蹲低/站高（TODO 1-6 的直接效果）
- **调试标记** — `base_height` 命令开启 `debug_vis` 后，Viser 中用 **橙色/青色球体** 显示目标与实际骨盆高度

本作业 **强烈建议** 使用 Viser 录屏作为蹲走性能（§ 11.1 30%）的佐证，原因有三：

1. **验证策略是否真正跟踪双指令**，而非只在固定指令下偶然成功
2. **培养调试习惯** — 训练 loss 下降不等于行为正确，play + 交互 slider 是更直观的验收方式

3. Viser 是 mjlab 生态的默认可视化入口，与 `train` 一样属于框架的常规用法

录屏可嵌入实验报告或作为 zip 附件（见 § 8.3）。

代码对应：uv run play ... --viewer viser → § 7.3；高度 slider 与球体标记 → TODO 1-2 采样/metrics + TODO 6 奖励生效后的行为表现。

与 实践3 的对比

	实践 3（HoST Sim2Sim）	实践 4（本作业）
仿真入口	手写 <code>deploy_mujoco_host_student.py</code>	mjlab <code>train</code> / <code>play</code>
策略来源	预训练 checkpoint 部署	自己训练 PPO 策略
可视化	MuJoCo 原生 viewer	Viser Web viewer + 指令 slider
核心技能	观测/动作空间映射	MDP 设计 + mjlab 配置

2.1 任务概述

在 mjlab 中加载 Unitree G1 模型，机器人在 平地 上同时跟踪两类指令：

- 1. 速度指令 (v_x, v_y, ω_z) — 与 baseline 速度任务相同
- 2. 骨盆绝对高度指令 $h_{cmd} \in [0.45, 0.80]$ m — 控制机器人蹲低或站高

训练成功后，策略应能：

- 按速度指令在平地上行走（前进、侧移、转向）
- 按高度指令调整骨盆（root/pelvis）的 世界坐标 z
- 在 Viser 可视化中，橙色/青色球体标记目标高度，机器人实际高度应跟随变化

2.2 指令空间（Command）

环境每 `resampling_time_range` 秒重新采样指令。联合指令向量可写为：

式 (1)

代码块

```
1  c_t = [ v_x, v_y, \omega_z, h_cmd ]
```

符号	配置键	采样方式	代码位置
----	-----	------	------

<code>(v_x, v_y, ω_z)</code>	<code>commands["velocity"]</code> <code>]</code>	<code>UniformVelocityComm</code> <code>andCfg</code>	<code>velocity_env_cfg.py</code>
<code>h_cmd</code>	<code>commands["base_height"]</code>	<code>UniformBaseHeightCommandCfg</code>	TODO 3+ <code>height_command.py</code> TODO 1-2

高度指令采样（TODO 1）：

式 (2)

代码块

```
1  h_cmd ~ Uniform(h_min, h_max),    h_min=0.45, h_max=0.80
```

高度跟踪 metrics（TODO 2）：每步累积

式 (3)

代码块

```
1  error_height ← error_height + |h_cmd - z_root| / max_command_step
```

其中 `z_root = root_link_pos_w[:, 2]` 为骨盆世界坐标高度。

代码对应：

式 (2) → `TODO 1 _resample_command()`；

式 (3) → `TODO 2 _update_metrics()`；

命令注册 → `TODO 3`。

2.3 观测空间（Observation）

策略网络分为 **Actor**（部署可用）与 **Critic**（训练专用，含特权信息）。

Actor 观测项（`velocity_env_cfg.py` 中 `actor_terms`，部分由 G1 配置扩展）：

观测项	含义	维度（约）
<code>base_lin_vel</code>	基座线速度（IMU，带噪声）	3
<code>base_ang_vel</code>	基座角速度（IMU，带噪声）	3
<code>projected_gravity</code>	投影重力	3

joint_pos	关节位置（相对 default）	29（G1）
joint_vel	关节速度	29
actions	上一时刻动作	29
command	速度指令（v_x，v_y，ω_z）	3
height_command	骨盆高度指令 h_cmd	1（TODO 4 接入）

Critic 额外特权观测：

观测项	含义	TODO
foot_height	足部离地高度	已实现
foot_air_time	足部腾空时间	已实现
foot_contact	足部是否接触地面	TODO 10
foot_contact_forces	接触力（log 压缩）	已实现

关键设计：Actor **必须** 观测到 height_command，否则策略无法知道当前目标蹲姿高度。TODO 4 通过 generated_commands 将 base_height 命令接入 actor/critic 观测组。

代码对应：

height_command 观测 → TODO 4；

foot_contact → TODO 10。

2.4 奖励设计（Reward）

跟踪类奖励统一采用 高斯（指数）形式：

式 (4)

代码块

1 r = exp(- error² / std²)

其中 std 已在 RewardTermCfg.params 中预填，你只需实现 error 计算与 return。

奖励项	误差定义	weight（已配置）	TODO
track_base_height	(h_cmd - z_root)²	1.0	5（接线）+ 6（实现）

track_linear_velocity	$ v_{xy_cmd} - v_{xy} ^2 + v_z^2$	2.0	7
track_angular_velocity	$(\omega_z_cmd - \omega_z)^2 + \omega_{xy} ^2$	2.0	8

高度跟踪奖励（TODO 6，核心）：

式 (5)

代码块

```
1 height_error = (h_cmd - z_root)^2
2 r_height = exp( - height_error / std^2 )
```

线速度跟踪（TODO 7）：

式 (6)

代码块

```
1 lin_vel_error = Σ(v_xy_cmd - v_xy_actual)^2 + v_z_actual^2
2 r_lin = exp( - lin_vel_error / std^2 )
```

角速度跟踪（TODO 8）：

式 (7)

代码块

```
1 ang_vel_error = (ω_z_cmd - ω_z_actual)^2 + Σ(ω_xy_actual)^2
2 r_ang = exp( - ang_vel_error / std^2 )
```

说明：TODO 7-8 与「蹲姿」无直接公式关系，但是行走所必需的基础奖励；高度任务在 baseline 速度 MDP 上叠加高度指令与奖励，速度跟踪逻辑仍需你补全。

2.4.1 奖励项四维分类（Task / Style / Reg / Penalty）

课件与评分要求奖励设计覆盖以下四个维度。请在代码注释与实验报告中对每一项标注所属维度及设计意图。

维度	英文	含义	本任务代表项
任务	Task	跟踪外部指令	track_linear_velocity、

			<code>track_angular_velocity</code> 、 <code>track_base_height</code> (TODO 5-8)
风格	Style	步态 / 姿态风格	<code>upright</code> 、 <code>pose</code> (<code>variable_posture</code>)、 <code>air_time</code>
正则	Reg	动作平滑、软约束	<code>action_rate_l2</code> 、 <code>soft_landing</code>
惩罚	Penalty	硬约束、不良行为惩罚	<code>dof_pos_limits</code> 、 <code>foot_clearance</code> 、 <code>foot_slip</code> 、 <code>foot_swing_height</code> 、 <code>body_ang_vel</code> 、 <code>angular_momentum</code> 等

完整 reward 配置见 `velocity_env_cfg.py` 与 `env_cfgs.py` (TODO 5 接入 `track_base_height`)。

评分对应：

§ 11.1 奖励设计合理性 (40%) 考察四维覆盖是否完整、Task 项公式是否正确、高度与速度奖励是否协调。

2.5 课程学习 (Curriculum)

速度指令采样范围随训练步数 **由易到难** 扩大 (TODO 9)：

阶段	触发 step	<code>lin_vel_x</code> 范围	<code>ang_vel_z</code> 范围
Stage 0	0	(-1.0, 1.0)	(-0.5, 0.5)
Stage 1	5000×24	(-1.5, 2.0)	(-0.7, 0.7)
Stage 2	10000×24	(-2.0, 3.0)	(保持 Stage 1)

`step` 以 **环境步** 为单位；`5000 * 24` 表示约 5000 次 PPO rollout (每次 24 步) 后进入下一阶段。

代码对应：

TODO 9 在 `velocity_env_cfg.py` 的 `curriculum["command_vel"]` 中补全 Stage 1、Stage 2。

2.6 MDP 数据流图

代码块

```
1 flowchart TD
2   heightCmd["height_command.py\nTODO 1-2"] --> envCfg["env_cfgs.py\nTODO 3-5"]
3   envCfg --> rewards["rewards.py\nTODO 6-8"]
4   velCfg["velocity_env_cfg.py\nTODO 9"] --> train["PPO train"]
5   rewards --> train
6   obsPriv["observations.py\nTODO 10"] --> train
7   train --> play["play + Viser"]
```

从理论到代码的闭环：

代码块

```
1 环境 reset
2   → [TODO 1] 采样 h_cmd (式 2)
3   → [TODO 2] 更新 error_height metrics (式 3)
4   → [TODO 3] 注册 base_height 命令
5   → [TODO 4] Actor/Critic 观测 height_command
6   → 策略推理 → 关节动作
7   → [TODO 6] r_height (式 5)
8   → [TODO 7-8] r_lin, r_ang (式 6-7)
9   → [TODO 5] 奖励管理器汇总
10  → PPO 更新
11  → [TODO 9] 扩大速度采样范围
12  → [TODO 10] Critic 读取 foot_contact 特权信息
```

接下来，将介绍环境配置与代码结构，并给出 10 个 TODO 的详细实现要求。

3. 环境配置

前置要求：已完成实践1 Linux + NVIDIA GPU 基础环境

本节从 **下载作业 zip** → **解压** → **安装依赖** → **自检** 逐步说明。按顺序完成即可开始 TODO。

一键流程概览（细节见各小节）：

代码块

```
1 # 1. 解压
2 mkdir -p ~/shenlan_hw4 && cd ~/shenlan_hw4
3 unzip /path/to/HW4_蹲姿行走作业.zip
4
5 # 2. 安装 uv (若未安装)
6 curl -LsSf https://astral.sh/uv/install.sh | sh && source $HOME/.local/bin/env
```

```
7
8 # 3. 安装 mjlab 依赖 (在 mjlab/ 目录, 需联网)
9 cd mjlab && uv sync --extra cu128
10
11 # 4. 自检
12 uv run list-envs
13 PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 uv run python -m pytest
  tests/test_velocity_task.py -q
```

说明：作业 zip 不含 预装虚拟环境（.venv）和训练日志，体积约 25 MB。PyTorch、MuJoCo、Warp 等大依赖会在你本机执行 `uv sync` 时 联网下载（首次约 3–8 GB，视网络速度需 10–30 分钟）。

3.1 系统要求

项目	要求
操作系统	Linux （Ubuntu 20.04+ 推荐；WSL2 可用但 Viser 需额外配置）
GPU	NVIDIA GPU （训练必需；建议 RTX 3060 及以上）
显卡驱动	支持 CUDA 12.x （ <code>cu128</code> 对应 CUDA 12.8 版 PyTorch）
磁盘空间	≥ 15 GB 可用（依赖 + 训练 checkpoint）
网络	首次安装需 联网 下载 Python 包
Python	3.10–3.13 （作业包指定 3.13，由 <code>uv</code> 自动管理，无需系统预装）

检查 GPU 与驱动（可选）：

代码块

```
1 nvidia-smi
```

若命令正常输出 GPU 型号与驱动版本，则硬件侧基本满足要求。

3.2 获取与解压作业包

- 从[深蓝学院《人形机器人运动控制》课程第 3 章](#)下载 `HW4_蹲姿行走作业.zip`
- 在终端进入你希望存放作业的目录，例如：

代码块

```
1  mkdir -p ~/shenlan_hw4
2  cd ~/shenlan_hw4
```

3. 解压（将 `/path/to/HW4_蹲姿行走作业.zip` 替换为实际路径）：

代码块

```
1  unzip /path/to/HW4_蹲姿行走作业.zip
```

4. 确认解压后的 **目录结构** 如下：

代码块

```
1  ~/shenlan_hw4/                                # 你选择的解压目录
2  └─ mjlab/                                       # 作业代码（所有命令在此目录下执行）
3      └─ pyproject.toml                         # 项目依赖定义
4      └─ uv.lock                                # 锁定版本（保证可复现安装）
5      └─ .python-version                       # Python 3.13
6      └─ src/mjlab/                             # 源码（含 10 个 TODO）
7      └─ tests/                                 # 测试
8      └─ scripts/                              # 辅助脚本
9      └─ typings/                              # 类型 stub
```

注意：

- zip 中 **没有** `.venv/`、`.git/`、`logs/`、`__pycache__/`，这些会在安装或运行后在本机生成
- 后续所有 `uv run train/play` 命令均在 `mjlab/` 目录下执行

3.3 安装 uv 包管理器

`mjlab` 使用 `uv` 管理 Python 版本与依赖。若尚未安装：

代码块

```
1  curl -LsSf https://astral.sh/uv/install.sh | sh
```

安装后 **重新打开终端**，或执行：

代码块

```
1  source $HOME/.local/bin/env # 或 source ~/.bashrc
```

验证：

代码块

```
1 uv --version
```

无报错即可进入下一步。

3.4 安装 mjlab 依赖

进入作业代码目录并同步依赖（需联网，首次较慢）：

代码块

```
1 cd ~/shenlan_hw4/mjlab          # 替换为你的实际路径
2 uv sync --extra cu128
```

参数	含义
uv sync	按 uv.lock 创建 .venv/ 并安装全部依赖
--extra cu128	安装 CUDA 12.8 版 PyTorch（GPU 训练必需）

安装过程中 uv 会自动：

- 1. 下载并安装 Python 3.13（若本机没有）
- 2. 创建虚拟环境 mjlab/.venv/
- 3. 安装 MuJoCo、PyTorch、NVIDIA Warp、Viser、RSL-RL 等

无 GPU、仅跑配置测试时（不能训练）：

代码块

```
1 uv sync --extra cpu
```

常见安装问题：

现象	处理
uv: command not found	完成 § 3.3，确认 \$HOME/.local/bin 在 PATH 中
下载 PyTorch 超时	检查网络；校园网可尝试代理或多次重试

CUDA / 驱动不匹配	升级 NVIDIA 驱动；或联系助教确认 GPU 型号
磁盘空间不足	清理旧 <code>.venv</code> ；确保 ≥ 15 GB 可用

3.5 快速自检

安装完成后，在 `mjlab/` 目录下依次执行：

Step 1 — 列出已注册任务

代码块

```
1 cd ~/shenlan_hw4/mjlab
2 uv run list-envs
```

预期输出包含：

代码块

```
1 Mjlab-Velocity-Flat-Unitree-G1
2 Mjlab-VelocityHeight-Flat-Unitree-G1
```

Step 2 — 运行 velocity 配置测试

代码块

```
1 PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 uv run python -m pytest
  tests/test_velocity_task.py -q
```

未完成 TODO 3-5 时，部分 height 相关测试 预期失败；若出现 `ModuleNotFoundError` 或 `uv run` 本身报错，说明环境未装好，回到 § 3.4 排查。

Step 3 — 确认 TODO 可定位

代码块

```
1 grep -rn "【作业 TODO" src/mjlab/tasks/velocity/ | head
```

应能看到 10 处 TODO 标记。

环境 OK 的标志：

- ☐ `uv run list-envs` 输出两个 G1 任务 ID
- ☐ `uv run python -c "import mjlab; import torch; print(torch.cuda.is_available())"` 输出 `True`（GPU 机器）

☐ 能在源码中搜到 【作业 TODO

全部通过后，即可阅读 § 6 作业任务 开始实现。

安装完成后，建议熟悉三条 **mjlab** 常用命令：

命令	作用
<code>uv run list-envs</code>	列出已注册任务 ID（确认环境装对）
<code>uv run train <Task-ID> ...</code>	启动 GPU 并行 PPO 训练
<code>uv run play <Task-ID> --viewer viser ...</code>	加载 checkpoint，浏览器中 Viser 交互验证

3.6 GPU 与显存建议

默认训练配置使用 **4096 个并行环境**：

代码块

```
1 uv run train Mjlab-VelocityHeight-Flat-Unitree-G1 --env.scene.num-envs 4096
```

GPU 显存	建议 <code>num-envs</code>
≥ 24 GB（如 RTX 4090）	4096
16 GB	2048
12 GB	1024（训练较慢）

4. 代码库结构

解压 `HW4_蹲姿行走作业.zip` 后，作业相关路径如下（以 `~/shenlan_hw4/` 为例）：

代码块

```
1 ~/shenlan_hw4/
2   └─ mjlab/
3       └─ src/mjlab/tasks/velocity/
4           └─ mdp/
5               └─ height_command.py      # TODO 1-2：高度指令采样与 metrics
6               └─ rewards.py             # TODO 6-8：跟踪奖励
7               └─ observations.py        # TODO 10：foot_contact 特权观测
8           └─ config/g1/
```

```

9      |   |   └─ env_cfgs.py          # TODO 3-5: G1 高度任务接线
10     |   |   └─ __init__.py         # 任务注册
11     |   └─ velocity_env_cfg.py      # TODO 9: 速度课程 + 共享 MDP 工厂
12     └─ HOMEWORK_TODO_INDEX.py      # TODO 索引 (grep 用)
13 └─ tests/test_velocity_task.py      # 自动评分参考
14 └─ src/mjlab/scripts/              # train, play, list-envs 入口

```

4.1 如何定位 TODO

在 `mjlab/` 目录下：

代码块

```

1  grep -rn "【作业 TODO" src/mjlab/tasks/velocity/
2  grep -rn "HOMEWORK_TODO_" src/mjlab/tasks/velocity/

```

每个 TODO 使用 `>>> HOMEWORK_TODO_N_START / <<< HOMEWORK_TODO_N_END` 标记边界。

每处 TODO 下方有中文提示；再下方 `# --- 实现提示 ---` 为 **非可执行** 的步骤/概念指引，**不可直接 copy-paste 运行**。请结合 §6 与 §2 理论自行实现；删除 `raise NotImplementedError` 后填入你的代码。

4.3 mjlab 配置层级（阅读导览）

完成 TODO 时，建议理解以下 **mjlab 配置层级**（只读，无需修改）：

代码块

```

1  任务 ID (如 Mjlab-VelocityHeight-Flat-Unitree-G1)
2    └─ config/g1/__init__.py          # 注册任务 → 指向 env_cfgs 工厂函数
3        └─ env_cfgs.py                # G1 专用：传感器、TODO 3-5 接线
4            └─ velocity_env_cfg.py    # 共享 MDP 模板：obs/reward/curriculum 默认值
5                └─ mdp/*.py           # 各 term 的具体实现（你的 TODO 1-2, 6-8,
10)

```

- `make_velocity_env_cfg()` — 所有 velocity 任务的 MDP 模板（观测项、奖励项、PPO 相关配置）
- `unitree_g1_flat_height_env_cfg()` — 在 G1 flat 配置 **之上叠加** 高度命令/观测/奖励 (TODO 3-5)
- **Manager** — 运行时按 cfg 实例化 CommandManager、ObservationManager、RewardManager 等

4.4 主任务与 baseline 对比

任务 ID	说明
Mjlab-Velocity-Flat-Unitree-G1	仅速度跟踪（baseline，无高度 TODO）
Mjlab-VelocityHeight-Flat-Unitree-G1	本作业主任务 — 速度 + 骨盆高度

5. MDP 与训练流程概览

5.1 训练流程（mjlab + Warp 并行）

代码块

```
1  uv run train Mjlab-VelocityHeight-Flat-Unitree-G1
2  |
3  |─ mjlab 加载 env cfg, Warp 在 GPU 上创建 4096 个并行 MuJoCo 实例
4  |─ 每步: CommandManager 采样/保持指令 → ObservationManager 计算 obs
5  |─ 每步: 策略推理 → 物理步进 → RewardManager 调用 track_* (TODO 6-8)
6  |─ 每 rollout: RSL-RL PPO 更新 Actor-Critic
7  |─ 定期保存 checkpoint → logs/rsl_rl/g1_velocity_height/<run>/
```

4096 个并行环境 意味着每步同时采集 4096 条 transition，这是 mjlab/Warp 相对「单环境 MuJoCo 脚本」的核心优势——同样的 wall-clock 时间内见到更多样本，人形 locomotion 策略通常在 **2-4 万 iter** 内可见可用行为。

5.2 Play 验证流程（Viser）

代码块

```
1  uv run play ... --viewer viser
2  |
3  |─ mjlab 加载 checkpoint 与 env cfg (单环境或少量 env)
4  |─ 启动 Viser 本地 Web 服务, 浏览器打开 3D 视图
5  |─ GUI: 速度 slider + 高度 slider (交互式改指令)
6  |─ debug_vis: 橙色/青色球体标记 base_height 目标/实际高度
7  |─ 录屏 (建议) → outputs/play_demo.mp4, 助评 §7.4 / §11.1
```

Viser 与训练共用同一套 env cfg，因此 play 中 slider 的行为 **直接反映** 你在 TODO 1-6 中的实现是否正确。若高度 slider 无效，优先排查 TODO 3-4 接线与 TODO 6 奖励。

5.3 学生需实现的 10 步在流程中的位置

流程阶段	对应 TODO
指令采样（reset / 定时重采样）	TODO 1, 2
环境配置（命令/观测/奖励注册）	TODO 3, 4, 5
每步奖励计算	TODO 6, 7, 8
训练过程中扩大速度范围	TODO 9
Critic 特权观测	TODO 10

6. 作业任务（10 个 TODO）

打开 `mjlab/src/mjlab/tasks/velocity/` 下各文件，找到 10 处 **【作业 TODO N/10】** 标记，按推荐顺序完成实现。

说明：本节 TODO 为实现指引与必做前提，帮助你搭好可训练的 MDP；最终成绩不按 TODO 逐项给分，而以 § 11 评分细则 中的奖励设计、蹲走性能、消融分析与代码规范四维评定。

建议顺序：TODO 1 → 2 → 3 → 4 → 5 → 6 → 7 → 8 → 9 → 10

优先级	TODO	主题	文件
核心	1-2	高度指令逻辑	<code>height_command.py</code>
核心	3-5	环境接线	<code>env_cfgs.py</code>
核心	6	高度跟踪奖励	<code>rewards.py</code>
配套	7-8	速度跟踪奖励	<code>rewards.py</code>
配套	9-10	课程 + 特权 obs	<code>velocity_env_cfg.py</code> , <code>observations.py</code>

TODO 1：高度指令均匀采样

目标文件： `height_command.py`

函数： `UniformBaseHeightCommand._resample_command()`

理论对应：式 (2)

简要提示：

- 在 `cfg.ranges.height` 给定的 `(h_min, h_max)` 范围内均匀随机采样
- 使用已创建的 `r = torch.empty(len(env_ids), device=self.device)`
- 写入 `self.height_command[env_ids, 0]`
- 删除 `raise NotImplementedError` 后填入实现

验证标准：

- 函数不再抛出 `NotImplementedError`
- 采样值落在 `[0.45, 0.80]` 内
- 训练 / play 时 Viser 中高度 slider 可正常工作

常见错误：

- 忘记指定 `env_ids` 索引，导致只更新了部分环境
 - 使用 `random.uniform` 而非 `torch` 向量化采样
-

TODO 2：高度跟踪误差 metrics

目标文件： `height_command.py`

函数： `UniformBaseHeightCommand._update_metrics()`

理论对应：式 (3)

简要提示：

- `actual_height = self.robot.data.root_link_pos_w[:, 2]`
- `height_error = torch.abs(self.height_command[:, 0] - actual_height)`
- 累积： `self.metrics["error_height"] += height_error / max_command_step`
- 更新： `self.metrics["target_height_mean"] = self.height_command[:, 0]`

验证标准：

- 训练日志中可看到 `error_height` 相关 metrics
- 无 `NotImplementedError`

常见错误：

- 使用 `.item()` 或 Python float 破坏批量张量运算
 - 忘记除以 `max_command_step` 导致 metrics 数值过大
-

TODO 3: 注册 base_height 命令

目标文件: `env_cfgs.py`

函数: `unitree_g1_flat_height_env_cfg()`

理论对应: 式 (1) 中 `h_cmd` 的配置入口

简要提示:

- 向 `cfg.commands` 注册 `"base_height"`, 使用 `UniformBaseHeightCommandCfg`
- 参考 `velocity_env_cfg.py` 中 `commands["velocity"]` 的注册模式
- `entity_name="robot"` 与 scene 中机器人实体一致; `ranges.height` 见 § 2.2
- `resampling_time_range` 控制高度指令重采样间隔

验证标准:

- `test_g1_velocity_height_task_has_commands` 通过
- `load_env_cfg("Mjlab-VelocityHeight-Flat-Unitree-G1").commands` 含 `"base_height"`

常见错误:

- 在 `unitree_g1_flat_env_cfg()` 而非 `unitree_g1_flat_height_env_cfg()` 中注册
 - 拼写错误: `base_heigh` / `height_base`
-

TODO 4: 接入 height_command 观测

目标文件: `env_cfgs.py`

函数: `unitree_g1_flat_height_env_cfg()`

理论对应: § 2.3 Actor 观测表

简要提示:

- 完善 `height_command_obs`: `params["command_name"]` 须与 TODO 3 注册的 `commands` 键一致
- 将 `height_command` 加入 **actor** 与 **critic** 的 `cfg.observations[...].terms`
- `func=envs_mdp.generated_commands` 从命令管理器读取当前指令

验证标准:

- `test_g1_velocity_height_task_has_observations_and_rewards` 通过
- `"height_command" in cfg.observations["actor"].terms`

常见错误:

- `command_name` 与 TODO 3 中注册的键不一致（必须均为 `"base_height"`）
 - 只接入 actor 忘记 critic
-

TODO 5: 注册 `track_base_height` 奖励项

目标文件: `env_cfgs.py`

函数: `unitree_g1_flat_height_env_cfg()`

理论对应: 式 (5) 的配置入口

简要提示:

- 向 `cfg.rewards` 注册 `"track_base_height"`, 使用 `RewardTermCfg`
- `func=mdp.track_base_height` 指向 TODO 6 将实现的函数
- `params` 含 `command_name` 与 `std`; `weight` 见 § 2.4 / 课件预填值

验证标准:

- `"track_base_height" in cfg.rewards`
- 训练启动后奖励项列表含 `track_base_height` (完成 TODO 6 后实际产生非 NaN 奖励)

常见错误:

- 在未完成 TODO 6 时期望训练 reward 正常 (会 `NotImplementedError`)
 - 修改 `weight/std` 导致与参考策略不一致 (本作业不要求调参)
-

TODO 6: 高度跟踪奖励 (核心)

目标文件: `rewards.py`

函数: `track_base_height()`

理论对应: 式 (5)

简要提示:

- `command = env.command_manager.get_command(command_name)` 已提供
- `actual_height = asset.data.root_link_pos_w[:, 2]`
- `height_error = torch.square(command[:, 0] - actual_height)`
- `return torch.exp(-height_error / std**2)`
- 返回 shape 为 `[num_envs]` 的一维张量

验证标准:

- 无 `NotImplementedError`

- 训练时 `track_base_height` 奖励为合理正值（约 0-1）
- Play 时改变高度 slider，骨盆高度跟随

常见错误：

- 使用 body frame 高度而非世界坐标 `root_link_pos_w[:, 2]`
 - 忘记平方：`abs` 而非 `square` 仍有一定效果但公式不符
-

TODO 7：线速度跟踪奖励

目标文件： `rewards.py`

函数： `track_linear_velocity()`

理论对应：式 (6)

简要提示：

- `actual = asset.data.root_link_lin_vel_b` 已提供（body frame）
- `xy_error = torch.sum(torch.square(command[:, :2] - actual[:, :2]), dim=1)`
- `z_error = torch.square(actual[:, 2])`（惩罚 vertical 漂移，指令 z 速度为 0）
- `lin_vel_error = xy_error + z_error`
- `return torch.exp(-lin_vel_error / std**2)`

验证标准：

- 无 `NotImplementedError`
- 训练能正常进行，机器人可行走

常见错误：

- 只跟踪 xy 忘记 z 漂移惩罚
 - 使用世界 frame 速度而非 `root_link_lin_vel_b`
-

TODO 8：角速度跟踪奖励

目标文件： `rewards.py`

函数： `track_angular_velocity()`

理论对应：式 (7)

简要提示：

- `actual = asset.data.root_link_ang_vel_b` 已提供

- `z_error = torch.square(command[:, 2] - actual[:, 2])` (yaw 角速度跟踪)
- `xy_error = torch.sum(torch.square(actual[:, :2]), dim=1)` (惩罚 roll/pitch 角速度)
- `ang_vel_error = z_error + xy_error`
- `return torch.exp(-ang_vel_error / std**2)`

验证标准:

- 无 `NotImplementedError`
- 训练时转向指令可跟踪

常见错误:

- 混淆 `command[:, 2]` 与 `actual[:, 2]` 的顺序
 - 对 xy 角速度也做指令跟踪 (应为惩罚接近 0, 而非跟踪非零指令)
-

TODO 9: 速度指令课程学习

目标文件: `velocity_env_cfg.py`

区域: `curriculum["command_vel"]`

理论对应: § 2.5 课程表

简要提示:

- 在 `velocity_stages` 列表中追加两个阶段 (Stage 0 已给出)
- Stage 1: `{"step": 5000 * 24, "lin_vel_x": (-1.5, 2.0), "ang_vel_z": (-0.7, 0.7)}`
- Stage 2: `{"step": 10000 * 24, "lin_vel_x": (-2.0, 3.0)}` (`ang_vel_z` 可省略, 沿用上一阶段)

验证标准:

- 配置可正常 `load_env_cfg`, 无语法错误
- 训练日志中后期速度指令范围扩大

常见错误:

- `step` 写为 `5000` 而非 `5000 * 24`
 - 覆盖 Stage 0 而非追加到列表
-

TODO 10: 足部接触特权观测 (Critic)

目标文件: `observations.py`

函数: `foot_contact()`

理论对应: § 2.3 Critic 特权观测

简要提示:

- `sensor_data.found` 已 assert 非 None
- 返回 `(sensor_data.found > 0).float()` — 形状 `[num_envs, num_feet]`
- 该观测 **仅 Critic 可见** (actor 观测组中无 `foot_contact`, 配置已写好)

验证标准:

- 无 `NotImplementedError`
- 训练 Critic 可正常收敛 (完整训练后 play 效果更稳定)

常见错误:

- 返回 bool 而非 float tensor
- 使用错误的 contact 字段 (应为 `found` 而非 `force`)

7. 运行与验证

7.1 单元 / 配置测试

完成 TODO 3-5 后, height 相关配置测试应通过:

代码块

```
1 cd mjlab
2 PYTEST_DISABLE_PLUGIN_AUTOLOAD=1 uv run python -m pytest
  tests/test_velocity_task.py -q
```

关键测试项:

测试函数	验证内容
<code>test_g1_velocity_height_task_has_commands</code>	TODO 3: <code>base_height</code> 命令
<code>test_g1_velocity_height_task_has_observations_and_rewards</code>	TODO 4-5: <code>height_command</code> obs + <code>track_base_height</code> reward

说明: 未完成 TODO 3-5 时, 上述测试 预期失败。全部 TODO 完成后应全部通过。

7.2 训练

代码块

```
1 cd mjlabs
2 uv run train Mjlab-VelocityHeight-Flat-Unitree-G1 --env.scene.num-envs 4096
```

- 日志目录: `logs/rsl_rl/g1_velocity_height/<timestamp>/`
- 建议训练 ≥ 20000 iter 再用于实验报告、蹲走性能验收与 checkpoint 附件
- 可用 TensorBoard 或终端打印观察 `error_height`、`track_base_height` 等指标

7.3 Play 验证 (Viser)

代码块

```
1 uv run play Mjlab-VelocityHeight-Flat-Unitree-G1 \
2   --checkpoint-file logs/rsl_rl/g1_velocity_height/<run>/model_29999.pt \
3   --viewer viser
```

命令启动后, 终端会打印 Viser 本地 URL (通常为 `http://localhost:8080`)。在浏览器中打开即可。

Viser 操作要点:

1. 在 GUI 面板找到 **velocity** 与 **base_height** 相关 slider
2. 先固定速度、只调高度 — 确认蹲低/站高 (验证 TODO 1, 4, 6)
3. 再固定高度、给非零速度 — 确认蹲姿行走
4. 观察骨盆处 **橙色 (实际)** / **青色 (目标)** 球体是否对齐
5. **强烈建议**录屏, 用于 § 7.4 蹲走性能验收 与 实验报告

7.4 蹲走性能验收 (评分用)

课件要求: 在 $h = 0.5\text{ m}$ 下稳定行走 $\geq 10\text{ s}$ 。按以下步骤自测 (助评 § 11.1 蹲走性能 30%) :

1. 启动 Viser play (§ 7.3), 加载自训练 checkpoint
2. 将 **base_height** slider 设为 **0.5 m**
3. 给非零前进速度 (如 `v_x  $\approx$  0.5–1.0 m/s`)
4. 观察机器人 **连续行走 $\geq 10\text{ s}$** , 不倒地, 骨盆高度接近 0.5 m
5. 在实验报告中记录: checkpoint iter、速度指令、是否达标; **建议附 Viser 录屏**

--	--

等级（自评参考）	现象
达标	$h = 0.5\text{ m}$ 下连续行走 $\geq 10\text{ s}$ ，无明显失稳
部分达标	能蹲走但 $< 10\text{ s}$ 或偶发倒地
未达标	无法蹲走或无法维持 0.5 m 高度

7.5 训练曲线导出（报告用）

实验报告须包含 **高度跟踪误差**、**速度跟踪误差**、**总回报** 三类曲线。数据来源：

曲线	来源	说明
高度跟踪误差	<code>error_height</code> metric	TODO 2 在 <code>height_command.py</code> 中实现；训练日志 / TensorBoard 中可见
速度跟踪误差	<code>track_linear_velocity</code> / <code>track_angular_velocity</code> 奖励 或相关 metrics	终端训练输出或 TensorBoard
总回报	Episode reward / <code>mean_reward</code>	<code>logs/rsl_rl/g1_velocity_height/<run>/</code> 下日志；若启用 W&B 可从网页导出

导出方式（任选其一）：

代码块

```
1  # 训练日志目录（具体文件名因 run 而异）
2  ls logs/rsl_rl/g1_velocity_height/<run>/
3
4  # 若配置了 TensorBoard
5  tensorboard --logdir logs/rsl_rl/g1_velocity_height/
```

将曲线 **截图嵌入实验报告** 即可；消融对比（§ 8.2）中不同方案也建议使用相同指标便于横向比较。

7.6 预期现象

- 机器人平地行走，同时跟踪速度指令
- 骨盆高度随 `base_height` 指令在 $0.45\text{--}0.80\text{ m}$ 范围内变化
- Viser 中橙色（实际） / 青色（目标）球体高度一致
- 终端无 `NotImplementedError`

7.7 调试表

现象	可能原因	排查方向
NotImplementedError	对应 TODO 未实现	<code>grep NotImplementedError</code> <code>src/mjlab/tasks/velocity/</code>
高度不跟踪	TODO 4 或 6 错误	检查 <code>command_name="base_height"</code> 与 <code>root_link_pos_w[:, 2]</code>
测试缺 <code>base_height</code>	TODO 3 未注册	运行 <code>pytest</code> 单测
训练 reward 全 NaN	TODO 6-8 公式错误	打印 reward term 中间值
机器人不行走	TODO 7-8 未实现	检查速度奖励是否抛出异常
checkpoint 无法 load	训练未完成或路径错误	确认 <code>model_*.pt</code> 存在

8. 提交要求

请将 奖励函数代码 与 实验报告 打包为 `用户名-实践4.zip` 提交（格式见 § 8.4）。

8.1 提交内容（必交）

序号	内容	格式	说明
1	奖励函数相关代码	Python 源文件	至少含 <code>rewards.py</code> （TODO 6-8）及 <code>env_cfgs.py</code> （TODO 5 接线）；建议提交完整 <code>mjlab/src/mjlab/tasks/velocity/</code> 目录。每项奖励须有注释说明设计意图与所属维度（Task/Style/Reg/Penalty）
2	实验报告	PDF 或 Markdown	结构见 § 8.2，建议命名 <code>用户名-实践4-报告.pdf</code>

8.2 实验报告结构（必含）

1. **奖励函数设计思路** — 按 Task / Style / Reg / Penalty 四维组织；说明蹲姿 + 速度双指令 MDP 的整体 reward 结构
2. **权重选择依据** — 对照 `velocity_env_cfg.py` / `env_cfgs.py` 中预填的 `weight`、`std`；解释为何如此配置（即使未改代码也须分析）
3. **消融实验** — 至少 **2 组** 不同设计方案对比，例如：
 - 方案 A：无 `track_base_height`（或 `weight = 0`）vs 方案 B：完整高度奖励（主方案）
 - 或不同 reward 组合 / 权重比例的分析或短训对比
 - 每组须有 **对比指标**（如 `error_height`、总回报、蹲走是否成功）与 **结论**
4. **训练曲线** — 至少包含：
 - 高度跟踪误差（`error_height`）
 - 速度跟踪相关误差或奖励
 - 总回报（episode reward）
 - 截图来源：终端日志 / TensorBoard / W&B（见 § 7.5）

8.3 建议附件（助评「蹲走性能 30%」）

以下 **非课件必交项**，但强烈建议提供，便于助教验收 § 11.1：

附件	说明
自训练 <code>model_*.pt</code>	复现实验与 play 验证
Viser 录屏 (mp4)	<code>h_cmd = 0.5 m</code> 下稳定行走 $\geq 10\text{ s}$ （见 § 7.4）；可嵌入报告或单独放入 zip

8.4 打包示例

代码块

```

1  cd ~/shenlan_hw4
2  zip -r 用户名-实践4.zip \
3    mjlabsrc/mjlabs/tasks/velocity/ \
4    用户名-实践4-报告.pdf \
5    outputs/play_demo.mp4 \
6    logs/rs1_rl/gl_velocity_height/<run>/model_29999.pt

```

后两项（录屏、checkpoint）为建议附件，可按实际情况取舍。

8.5 提交前检查清单

- ☐ 10 个 TODO 均已实现，无 NotImplementedError（计分前提，见 § 11.3）
- ☐ 奖励代码中 每项 reward 有设计意图注释（含 Task/Style/Reg/Penalty 标注）
- ☐ 实验报告含 设计思路、权重依据、消融 ≥ 2 组、训练曲线 四部分
- ☐ 已完成 $h = 0.5\text{ m}$ 蹲走 $\geq 10\text{ s}$ 自测（建议附录屏）
- ☐ zip 内含 代码 + 实验报告（必交）
- ☐ 压缩包命名符合 用户名-实践4.zip 规范

评分标准见评分细则。

9. 评分细则

总分：100 分

本作业按课件 **四个维度** 评定，与 § 8 提交要求 对齐。助教依据 **奖励代码 + 实验报告** 综合打分；checkpoint / Viser 录屏作为蹲走性能的辅助佐证。

11.1 分值分配

评分项	分值	考察内容
奖励设计合理性	40%	覆盖 Task / Style / Reg / Penalty 四维；公式与注释清晰；高度 + 速度 Task 奖励协调
蹲走性能	30%	$h = 0.5\text{ m}$ 下稳定行走 $\geq 10\text{ s}$ (Viser 录屏或报告内说明 + 曲线佐证)
消融分析质量	20%	≥ 2 组设计方案；有对比指标 / 曲线 / 结论
代码规范与注释	10%	命名与结构； 每项奖励的设计意图注释 （含四维标注）
合计	100%	—

11.2 各维度评分标准

奖励设计合理性（40%）

得分	标准

36-40 分	Task/Style/Reg/Penalty 四维覆盖完整；TODO 6-8 公式正确；报告与代码注释一致；高度与速度 reward 分工清晰
20-35 分	四维均有涉及但某维分析偏薄；或 Task 公式 minor 错误；权重依据论述不完整
0-19 分	缺整维分析；或 <code>NotImplementedError</code> 未消除；reward 设计明显不合理

蹲走性能（30%）

得分	标准
27-30 分	h = 0.5 m 下连续稳定行走 ≥ 10 s，有录屏或报告内清晰说明测试条件与结果
15-26 分	能蹲走但时长不足 10 s，或高度跟踪误差偏大
0-14 分	无法蹲走 / 无法维持 0.5 m / 无性能说明且训练曲线显示未收敛

消融分析质量（20%）

得分	标准
18-20 分	≥ 2 组方案对比清晰；有定量指标或曲线；结论与设计理论一致
10-17 分	有两组对比但指标或结论较粗糙；或仅定性分析
0-9 分	无消融 / 仅一组 / 与 reward 设计无关

代码规范与注释（10%）

得分	标准
9-10 分	代码可读；TODO 实现处及 reward 项均有设计意图注释；无冗余 dead code
5-8 分	能运行但注释不足或四维标注缺失
0-4 分	代码混乱 / 大量未注释粘贴 / 仍含 <code>NotImplementedError</code>

11.3 计分前提

以下情况影响评定上限或整体不合格：

情况	处理
缺 实验报告	整体 不合格 或总分上限 ≤ 40 分
10 个 TODO 未全部实现（含 <code>NotImplementedError</code> ）	奖励设计与 蹲走性能 项上限各扣至 50%；代码规范 ≤ 4 分
无录屏但报告有 $h = 0.5\text{ m}$ 测试说明 + 曲线	可正常评 蹲走性能，凭证据给分
仅交代码、无报告	不予完整评分

11.4 提交物要求（计分前提）

以下两项 缺一不可：

- 1. 奖励函数相关代码 — 含 TODO 实现的 Python 源文件，reward 注释完整
- 2. 实验报告 — PDF 或 Markdown，含 § 8.2 四部分

打包格式： 用户名-实践4.zip

建议附件：checkpoint、Viser 录屏（ $h = 0.5\text{ m}$ 蹲走 $\geq 10\text{ s}$ ）

10. FAQ

Q1：为什么要观测 `height_command`？

A：策略是 指令跟踪 型 RL：Actor 必须知道当前目标高度 `h_cmd` 才能输出合适的蹲姿动作。若只观测本体状态而不观测指令，MDP 变为「盲猜」目标，无法训练。

Q2： `track_base_height` 与 `velocity` 奖励是什么关系？

A：高度奖励（TODO 6）是本作业的 核心新增（Task 维）；线速度/角速度奖励（TODO 7-8）是 行走能力 的基础（Task 维）。两者加权求和构成总 reward。须在实验报告中按 § 2.4.1 阐述各维设计意图与权重依据。

Q3：TODO 7-8 与蹲姿有什么关系？

A：无直接公式关系，但没有速度跟踪奖励，机器人无法稳定行走，蹲姿行走任务也无法完成。可以理解为在 baseline 速度 MDP 上 叠加 高度维度。

Q4：checkpoint 和录屏必须交吗？

A：课件必交为奖励代码 + 实验报告。checkpoint 与 Viser 录屏为 建议附件，用于助评「蹲走性能 30%」；若无录屏，须在报告中用文字 + 曲线说明 $h = 0.5\text{ m}$ 蹲走是否达标。须为 自训练 策略，不可

用他人 checkpoint 冒充。

Q5: pytest 报 ROS 插件错误怎么办？

A: 使用 `PYTEST_DISABLE_PLUGIN_AUTOLOAD=1` 前缀运行 pytest（见 § 3.5）。

Q6: zip 里没有 `.venv`，是正常的吗？

A: **正常**。作业包只含源码与 `uv.lock`，不含虚拟环境（否则体积达数 GB）。解压后必须在 `mjlab/` 下执行 `uv sync --extra cu128` 生成本地 `.venv/`。

Q7: 解压后应该 `cd` 到哪个目录？

A: 所有 `uv run train/play/list-envs` 命令均在 `mjlab/` 目录执行。

Q8: 为什么要学 mjlab，而不是继续写 MuJoCo 脚本？

A: 作业 2 的脚本式部署适理解 **单策略、单环境** 的推理链路；本作业的 **RL 训练** 需要大规模并行采样与 MDP 模块化配置，手写成本过高。mjlab 将 MuJoCo 精度与 Warp 并行结合，且被人形 locomotion 社区广泛采用——本作业让你用最小任务量熟悉这一 **工业/科研界常用栈**，而不仅是完成一次公式推导。

Q9: Viser 打不开的常见原因？

A: 检查防火墙是否拦截本地端口；确认 `--viewer viser` 参数已加；在无图形界面的远程服务器上，可能需要 SSH 端口转发（如 `ssh -L 8080:localhost:8080`）后在本地浏览器访问。

11. 参考资料

1. mjlab 官方仓库

<https://github.com/mujocolab/mjlab> — 框架设计与任务注册方式

2. Viser 可视化

<https://github.com/nerfstudio-project/viser> — Web 3D viewer；本作业通过 `play --viewer viser` 调用

3. TODO 快速索引

`mjlab/docs/HOMEWORK_TODO.md`

4. RSL-RL / PPO

本作业使用 on-policy PPO 训练；checkpoint 格式为 `model_{iter}.pt`

5. uv 包管理器

<https://docs.astral.sh/uv/> — 安装与 `uv sync` 用法

6. MuJoCo 文档

<https://mujoco.readthedocs.io/> — 物理引擎与接触建模原理

7. NVIDIA Warp

<https://github.com/NVIDIA/warp> — GPU 并行计算；mjlab 用它批量驱动 MuJoCo 实例

如有疑问，请在课程群联系助教。祝训练顺利，期待看到你的 G1 蹲姿行走！